STEP 1: DATA INGESTION

The Dataset Universe

Our training data isn't a perfect reflection of reality. It's heavily skewed towards common procedures.

DATA INSIGHT

In a dataset of 100,000 claims, only ~150 might represent this specific rare surgery. This creates a **CLASS IMBALANCE RATIO OF 1:666**

.

99.8%
Common Claims

0.2%
Rare Surgeries

What the Model Actually Learns

Input Stream (99.8%)

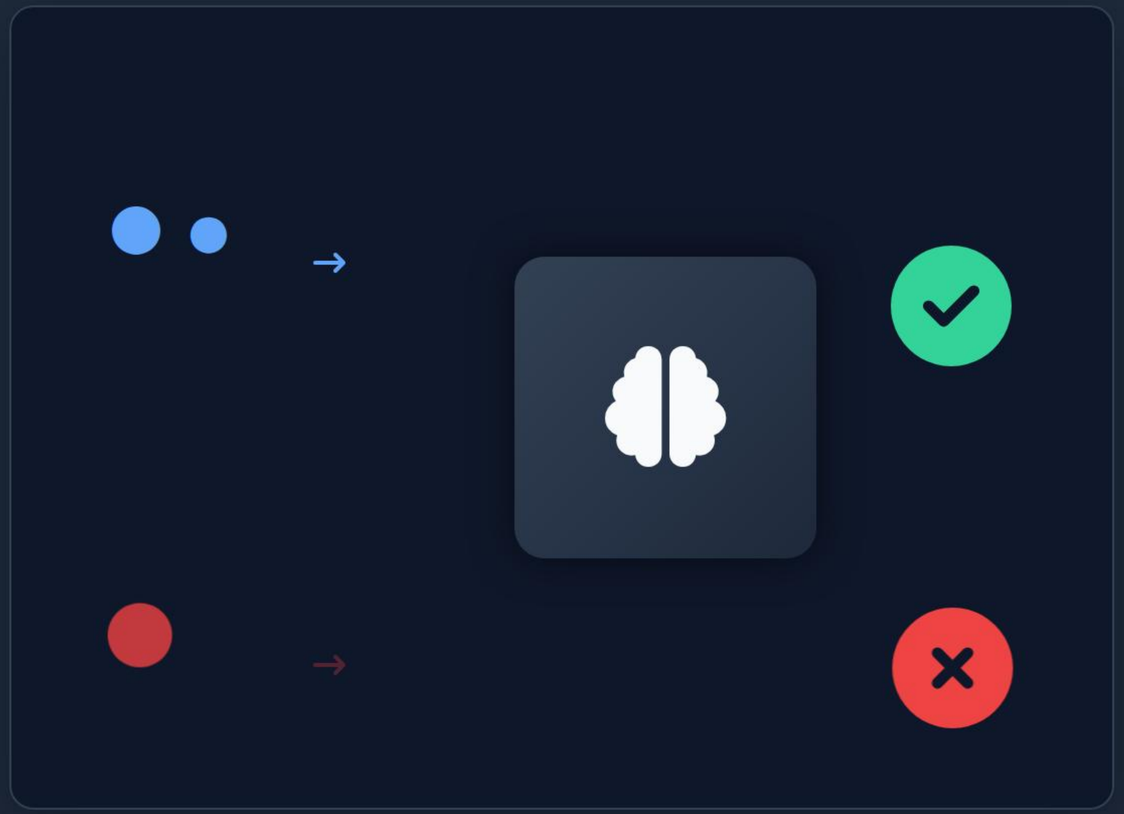
The model is flooded with "Blue Dot" examples. It learns these patterns instantly.

Optimization

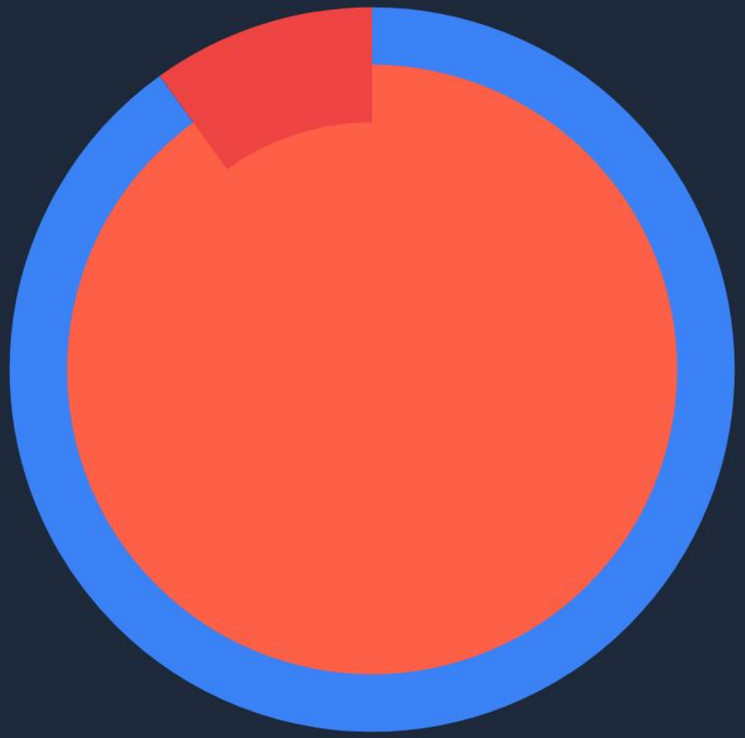
To minimize global error, the model decides: *"It's safer to guess 'Common' every time."*

⚠ THE CONSEQUENCE

The rare "Red Dot" claims are treated as statistical noise or anomalies, leading to incorrect rejections.



The "Accuracy Paradox"



Why Global Metrics Lie

Common Claims (90%)

Model Accuracy: **99%** (Globally acceptable)

Rare Surgeries (10%)

Model Accuracy: **< 40%** (Failure state)

The model looks successful on paper, but it is **failing** the patients who need specialized care.

Three Pillars of Risk

Data Imbalance

The sheer volume of common claims drowns out the signal from rare procedures, making learning difficult.

Under-representation

Key features unique to rare surgeries (e.g., specific orphan drugs) may be pruned by the model as "irrelevant."

Regulatory Risk

FDA & EU AI Act

New regulations specifically target "bias against subgroups." Non-compliance carries heavy fines.

Real World Impact

THE PATIENT

"I needed this life-saving procedure. Why was it auto-rejected as 'experimental' just because it's rare?"

→ Delayed Treatment
→ Financial Distress

THE MODEL

"I was optimized for global efficiency. I minimized error by assuming this pattern didn't exist."

→ Need for Subpopulation Testing
→ Need for Human-in-the-loop

The Remediation Checklist

We must move from "Aggregate Evaluation" to "Stratified Evaluation." Here is the MLOps protocol for rare events:

</> TECHNICAL TIP

Use **SMOTE** (Synthetic Minority Over-sampling Technique) to artificially boost the signal of rare claims during training.

Stratified Metrics

Measure Precision/Recall for the "Rare" slice specifically.

Cost-Sensitive Learning

Penalize the model 10x more for getting a rare claim wrong.

Anomaly Detection Fallback

Route low-confidence rare predictions to human reviewers.

Executive Summary

1
The Blind Spot

Models naturally bias towards majority data. Rare surgeries are invisible to standard training loops.

2
The Risk

High global accuracy hides **catastrophic failure** on rare subpopulations, leading to ethical and legal liability.

3
The Solution

Implement **Subpopulation Evaluation** and cost-sensitive training to ensure fairness for every patient.

Q4: Why is stress testing necessary for ML models in production? Suggest at least two methods to make the API stable under heavy claim-submission load.

How the System Actually Fails (Why Stress Testing Matters)

1. **Non-linear p95/p99 latency spikes**
   - ML APIs don’t degrade gradually.
   - At high concurrency, latency jumps suddenly (e.g., 200ms → 3 seconds).
2. **Memory pressure → model server reloads**
   - Large model artifacts cause reloading, blocking all inference threads.
3. **CPU spikes from heavy matrix ops**
   - Vectorized operations overwhelm CPU cores under concurrency.
4. **Python GIL + garbage collection freezes**
   - Thread contention + GC pauses → entire service hangs.
5. **Full inference stack collapse**
   - Preprocessing, inference, and post-processing become blocked, causing timeouts.

Purpose of stress testing:
To force these failures in staging so they never happen in production.

Process 1

Asynchronous Queue + Batch Workers (Fixes the Request Layer)

Why the request layer failed:

- Synchronous API tried to run inference for every incoming request
- Heavy load (5,000/min) caused threadpool exhaustion → API timeouts
- Feature extraction + model calls piled up → complete collapse

How this method works (using Celery + Redis / AWS SQS / RabbitMQ):

- API quickly pushes each claim into a queue instead of processing it
- Background workers pull claims from the queue
- Workers run inference in batches (e.g., 32–64 claims at once)
- API stays responsive; workers handle all heavy ML computation

Why this stabilizes the system:

- Absorbs sudden traffic spikes
- Prevents API overload and timeouts
- Enables batching → higher inference throughput
- Allows safe, controlled concurrency
- Protects the entire ML pipeline from collapse

Process 2

Why the inference layer failed:

- Model served through normal Python APIs → slow, inefficient under load
- No batching → every claim triggered a separate inference call
- Python GIL + GC pauses → unpredictable p95/p99 latency
- Heavy NumPy/Pandas preprocessing overwhelmed CPU
- Model reloads occurred under memory pressure

How Optimized Model Serving Solves This:

Using TorchServe / TensorFlow Serving / TensorRT provides:

- Warm-loaded model instances → no repeated reloads under load
- Built-in batching support (process multiple claims in a single inference call)
- High-performance C++ backends → avoids Python GIL limitations
- Optimized CPU/GPU kernels → efficient tensor execution
- Stable, predictable p95/p99 latency even at high concurrency
- Higher throughput without needing new hardware
- Better parallelization and controlled worker processes

Why this complements the async queue solution:

- **Process 1** protects the API layer from overload
- **Process 2** increases the actual processing speed of inference
- Together, they prevent both:
- ✓ API timeouts
- ✓ Worker slowdowns and queue buildup